# Reinforcement Learning

Hyoung Joon, Kim

July 15, 2026

# Contents

# Chapter 1

# Introduction

This document is a set of notes of algorithms of reinforcement learning and theories of reinforcement learning. Following books were mainly used for studying algorithms and theories.

- [Sut20];
- [Gra20];
- [Zha25];

# Part I

# Algorithms

# Chapter 2

# Dynamic Programming

## 2.1 Introduction

This chapter discuss and dyamic programming algorithms for finding optimal policies. Since these are dynamic prgramming algorithms, a perfect system model is required; this mean that it is rarely used in practice, but these algorithms important foundations of the model-free algorithms. Three algorithms will be presented;

- Value Iteration: The algorithms suggested by the contraction mapping theorem 9.5.0.1;
- Policy Iteration:
- Trucated Policy Iteration: The unified algorithm which includes value iteration and policy iteration as its special cases.

## 2.2 Value Iteration

### 2.2.1 Main Idea

The main idea of **value iteration** algorithms is the algorithms suggested in Theorem 9.5.0.1. In particular, the algorithms is

$$v_{k+1} = \max_{\pi \in \Pi} (r_\pi + \gamma P_\pi v_k), \quad k = 0, 1, 2, \dots \tag{2.1}$$

Breaking down, this iterative algorithm has two steps;

1. *Policy Update*: The algorithm finds a policy $\pi_{k+1}$ that solves following optimization problem;

$$\arg\max_{\pi} (r_\pi + \gamma P_\pi v_k) \tag{2.2}$$

2. *Value Update*: With the policy $\pi_{k+1}$, the algorithm calculates a new state value function $v_{k+1}$;

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k \tag{2.3}$$

and this $v_{k+1}$ is used in next iteration.

### 2.2.2 Pseudocode

**Algorithm 1:** Value Iteration Algorithm

**Input** : $p(s', r|s, a)$ ..... the dynamics of the system
$v$ ..... initial guess
**Output:** $v^*, \pi^*$ ..... optimal state value and optimal policy

$\pi \leftarrow$ greedy policy derived from $v$
**repeat**
  **for** $s \in \mathcal{S}$ **do**
    **for** $a \in \mathcal{A}(s)$ **do**
      $q(s, a) \leftarrow \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v(s')$
    `// policy update`
    $\pi(a|s) = \begin{cases} 1 & \text{if } a = \arg\max_a q(s, a) \\ 0 & \text{otherwise} \end{cases}$
    `// value update`
    $v(s) = \max_a q(s, a)$
**until** convergence of $v$
**return** $v$, $\pi$

Note that the value update of Algorithm 1 is simply substituting the original state values with maximum action values since we are using greedy policy.

### 2.2.3 Convergence

The convergence can be proved with Theorem 9.4.1.1 and Theorem 9.4.2.1.

## 2.3 Policy Iteration

### 2.3.1 Main Idea

The **policy iteration** is based on the policy improvement theorem. Like value iteration, the policy iteration consists of two steps:

1. *Policy Evaluation*: For a given policy $\pi_k$, we calculate the corresponding state values by solving following Bellman equation:
$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k} \tag{2.4}$$
2. *Policy Improvement*: Using the computed state value $v_k$, we obtain the improved policy as
$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_{\pi_k}) \tag{2.5}$$
and here is where the policy improvement theorem is applied.

The difference is that the value iteration is that instead of taking a state value and computing the policy from it, the policy iteration takes a policy and compute the value corresponding to the policy.

Also, note that policy iteration has another iterative algorithm inside it: the policy evaluation uses iterative algorithm for solving the Bellman equation $v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$.

### 2.3.2 Pseudocode

**Algorithm 2:** Policy Iteration Algorithm

**Input :** $p(s', r|s, a)$ ..... the dynamics of the system
$\pi$ ..... initial guess

**Output:** $v^*, \pi^*$ ..... optimal state value and optimal policy

**repeat**
  `// policy evaluation`
  $v' \leftarrow$ arbitrary initial guess
  **repeat**
    **for** each state $s \in \mathcal{S}$ **do**
      $v'(s) \leftarrow \sum_a \pi(a|s) \Big[ \sum_r p(r|s, a) r + \gamma \sum_{s'} p(s'|s, a) v'(s') \Big]$
  **until** convergence of $v'$
  $v \leftarrow v'$
  `// policy improvement`
  **for** each state $s \in \mathcal{S}$ **do**
    **for** each action $a \in \mathcal{A}(s)$ **do**
      $q(s, a) \leftarrow \sum_r p(r|s, a) r + \gamma \sum_{s'} p(s'|s, a) v(s')$
    $a^* \leftarrow \arg\max_a q(s, a)$
    $\pi(a|s) \leftarrow \begin{cases} 1 & \text{if } a = a^* \\ 0 & \text{otherwise} \end{cases}$
**until** convergence of $v$
**return** $v$, $\pi$

### 2.3.3 Convergence

TODO: convergence proof of policy iteration theorem. This will be done after studying the operator version of Bellman Equation.

## 2.4 Truncated Policy Iteration

### 2.4.1 Main Idea

The **truncated policy iteration** is a more general algorithm including value iteration and policy iteration as its special cases. The algorithm has only one difference with policy iteration: in the policy evaluation step, the truncated policy iteration iterates only finite times, even when the convergence condition is not met. Truncated policy iteration algorithm is one of the instances of **general policy iteration**, which is an abstract concept of interleaving evaluation and improvement.

The perspective of looking at the truncated policy iteration as the generalization of policy iteration and value iteration can

be derived from the similiarity between value iteration and policy iteration. With the assumption that $v_{\pi}^{(0)} = v_0 = v_{\pi_0}$,

$$v_{\pi_1}^{(0)} = v_0 \tag{2.6}$$

$$\text{value iteration} \leftarrow v_1 \longleftarrow v_{\pi_1}^{(1)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(0)} \tag{2.7}$$

$$v_{\pi_1}^{(2)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(1)} \tag{2.8}$$

$$v_{\pi_1}^{(3)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(2)} \tag{2.9}$$

$$\vdots \tag{2.10}$$

$$\text{truncated policy iteration} \leftarrow \bar{v}_1 \longleftarrow v_{\pi_1}^{(j+1)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(j)} \tag{2.11}$$

$$\vdots \tag{2.12}$$

$$\text{policy iteration} \leftarrow v_{\pi_1} \longleftarrow v_{\pi_1}^{(\infty)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(\infty)} \tag{2.13}$$

$$\tag{2.14}$$

While value iteration computes only once and update the state value and policy iteration iterates until convergence and use it as state value of policy, the truncated policy iteration iterates only finite number of times.

### 2.4.2 Pseudocode

**Algorithm 3:** Truncated Policy Iteration Algorithm

**Input** : $p(s', r|s, a)$ ..... the dynamics of the system
$\pi$ ..... initial guess

**Output:** $v^*, \pi^*$ ..... optimal state value and optimal policy

**repeat**
  `// policy evaluation`
  $v' \leftarrow$ arbitrary initial guess
  **for** $k = 0, 1, \ldots, k_{\text{truncate}} - 1$ **do**
    **for** each state $s \in \mathcal{S}$ **do**
      $v^{(k+1)}(s) \leftarrow \sum_a \pi(a|s) \Big[ \sum_r p(r|s, a) r + \gamma \sum_{s'} p(s'|s, a) v^{(k)}(s') \Big]$
  $v \leftarrow v^{(k_{\text{truncate}})}$
  `// policy improvement`
  **for** each state $s \in \mathcal{S}$ **do**
    **for** each action $a \in \mathcal{A}(s)$ **do**
      $q(s, a) \leftarrow \sum_r p(r|s, a) r + \gamma \sum_{s'} p(s'|s, a) v(s')$
    $a^* \leftarrow \arg\max_a q(s, a)$
    $\pi(a|s) \leftarrow \begin{cases} 1 & \text{if } a = a^* \\ 0 & \text{otherwise} \end{cases}$
**until** convergence of $v$
**return** $v$, $\pi$

### 2.4.3 Convergence

TODO: Convergence proof of truncated policy iteration.

# Chapter 3

# Monte Carlo Methods

## 3.1 Introduction

This chapter introduces a **Monte Carlo methods** for reinforcement learning. These algorithms all starts from trying to obtain the optimal policy by solving the Bellman optimality equation. The difference of pure dynamic programming algorithms and Monte Carlo methods is that the latter do not require a perfect dynamics of the model, i.e., it is *model-free*.

Then, how do we get the optimal policy without a perfect model? We use the data that the model gives us. Using the data, we can convert the policy iteration algorithm 2 to the model-free version. Specifically, we estimate the action value $q(s,a) = \mathbb{E}[G_t|S_t = a, A_t = a]$ using Monte Carlo sampling.

## 3.2 Monte Carlo Basic

### 3.2.1 Main Idea

The **Monte Carlo basic** is the simplest Monte Carlo method algorithm. Suppose that we run the agent starting from state-action pair $(s,a)$, following policy $\pi$, and let $g_\pi^{(i)}(s,a)$ be the return of the episode. Then, we can approximate the action value $q_\pi(s,a)$ as following:

$$q_\pi(s,a) = \mathbb{E}[G_t|S_t = s, A_t = a] \approx \frac{1}{n}\sum_{i=1}^{n} g_\pi^{(i)}(s,a). \tag{3.1}$$

If the number $n$ is sufficiently large, the approximation will be sufficiently accurate. This can be guaranteed due to the law of large numbers.

### 3.2.2 Pseudocode

**Algorithm 4:** Monte Carlo Basic

**Input :** $\pi_0$ ..... initial guess
**Output:** $\pi^*$ ..... optimal policy

**repeat**
  **foreach** $s \in \mathcal{S}$ **do**
    **foreach** $a \in \mathcal{A}(s)$ **do**
      `// policy evaluation`
      collect sufficiently many episodes starting from $(s, a)$, following policy $\pi_k$
      $q_{\pi_k}(s, a) \leftarrow$ the average return of all episodes starting from $(s, a)$
  `// policy improvement`
  $\pi_{k+1}(a|s) = \begin{cases} 1 & \text{if } a = \arg\max_{a'} q_{\pi_k}(s, a') \\ 0 & \text{otherwise} \end{cases}$
**until** convergence of $\pi$
**return** $\pi$

### 3.2.3 Convergence

### 3.2.4 Pros and Cons

Cons:

- Very inefficient sample usage. Due to this property, MC-Basic is not used in practical. To deal with this, we use MC with exploring start, which is presented in next section.

## 3.3 Monte Carlo with Exploring Starts

### 3.3.1 Main Idea

The MC-Basic algorithms is not sample-efficient, since it only updates only q value of one state-action pair each episode. To deal with this low efficiency, we use all the *visit* to the state-action pair in each episode.

Suppose that the agent has gone through following trajectory:

$$s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} s_3 \xrightarrow{a_3} s_4 \xrightarrow{a_4} \dots \tag{3.2}$$

To increase the sample efficiency, we use the visits to the state-action pairs in the trajectory. That is, using the above episode, we not only use $(s_1, a_1)$ for estimating $q(s_1, a_1)$ but use $(s_2, a_2)$ for estimating $q(s_2, a_2)$, and so on. There are two different ways to do this:

- The first way is to only use the state-action pairs when it is first encountered. This is called *first-visit method*;
- The second way is to use all the visit to the state-action pairs. This is called *every-visit method*.

Also, we update the estimate of q values after each episode ends. This falls into the scope of *generalized policy iteration*, since we are using the value approximation even it is not so accurate.

### 3.3.2 Pseudocode

**Algorithm 5:** Monte Carlo with Exploring Starts (every-visit)

**Input :** $\pi_0$ ..... initial policy
**Output:** $\pi^*$ ..... optimal policy

`// initialize`
**foreach** valid state-action pair $(s, a)$ **do**
  $\pi \leftarrow \pi_0$
  $q(s, a) \leftarrow$ initial value
  `// holds the sum of returns of (sub)episode starting from` $(s, a)$
  $\text{Ret}(s, a) \leftarrow 0$
  `// counts the number of visits of` $(s, a)$
  $\text{N}(s, a) \leftarrow 0$
**repeat**
  select a starting valid state-action pair $(s, a)$, ensuring that all the pairs can be possibly selected
  generate an episode starting from $(s, a)$, following current policy $\pi$: $s_0, a_0, r_1, s_1, a_1, \ldots, s_{T-1}, a_{T-1}, r_T$
  $G \leftarrow 0$
  **for** $t = T-1, T-2, \ldots, 0$ **do**
    $G \leftarrow r_{t+1} + \gamma G$
    $\text{Ret}(s, a) \leftarrow \text{Ret}(s, a) + G$
    $\text{N}(s, a) \leftarrow \text{N}(s, a) + 1$
    `// policy evaluation`
    $q(s_t, a_t) \leftarrow \frac{\text{Ret}(s,a)}{\text{N}(s,a)}$
    `// policy improvement`
    $$\pi_{k+1}(a|s) = \begin{cases} 1 & \text{if } a = \arg\max_{a'} q_{\pi_k}(s, a') \\ 0 & \text{otherwise} \end{cases}$$
**until** convergence

### 3.3.3 Implementation Details

The exploring-start condition can be achieved by random start state.

### 3.3.4 Pros and Cons

Pros:

- Better sample efficiency then MC-Basic;
- All state-action pairs can be visited due to the exploring-start, and this makes the estimation more accurate.

Cons:

- Only applicable to episodic tasks, which is a common drawback for all MC methods. This property leads to following problem, too;
- High variance since the return holds all the noises and randomnesses, and the return is used for update;
- If the exploring start condition cannot be satisfied, the MC with Exploring Starts cannot be used. This is common in real-world problems. To deal with this, another algorithms without exploring starts will be introduced next section;

# Chapter 4

# Policy-based Methods

## 4.1 Introduction

**Policy gradient method**, unlike value-based methods, tries to predict the optimal policy itself, instead of solving the Bellman equation. It has many advantages over value-based methods; It is more efficient in handling large state-action spaces, and has stronger generalization abilities.

## 4.2 REINFORCE

### 4.2.1 Main Idea

**REINFORCE** is the most basic form of policy gradient method. It's the vanilla policy gradient method.

### 4.2.2 Objective Function

REINFORCE uses following objective function;

$$J(\pi_{\boldsymbol{\theta}}) = \mathbb{E}_{\tau\sim\pi_{\boldsymbol{\theta}}}[G_t] = \mathbb{E}_{\tau\sim\pi_{\boldsymbol{\theta}}}\left[\sum_{t=0}^{T-1}\gamma^t r_{t+1}\right]. \tag{4.1}$$

We use some tricks to compute the gradient of $J(\pi_{\boldsymbol{\theta}})$ Then we get

$$\nabla_{\boldsymbol{\theta}} J(\pi_{\boldsymbol{\theta}}) = \mathbb{E}_{\tau\sim\pi_{\boldsymbol{\theta}}}\left[\sum_{t=0}^{T-1} G_t \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_t|s_t)\right]. \tag{4.2}$$

Now we can use this to update the parameter $\boldsymbol{\theta}$. Since we cannot compute the exact expectation, we use Monte Carlo sampling to approximate $\nabla_{\boldsymbol{\theta}} J(\pi_{\boldsymbol{\theta}})$ as following:

$$\nabla_{\boldsymbol{\theta}} J(\pi_{\boldsymbol{\theta}}) \approx \sum_{t=0}^{T-1} G_t \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_t|s_t) \tag{4.3}$$

### 4.2.3 Pseudocode

**Algorithm 6:** REINFORCE

**Input** : $\boldsymbol{\theta}$ ..... initial parameter
$\gamma$ ..... discount rate
$\alpha$ ..... learning rate

**Output:** $\pi_{\boldsymbol{\theta}}$ ..... learned policy

initialize policy $\pi_{\boldsymbol{\theta}}$
**repeat**
  generate episode $\{s_0, a_0, r_1, \ldots, s_{T-1}, a_{T-1}, r_T\}$ following $\pi_{\boldsymbol{\theta}}$
  **for** $t = 0, 1, \ldots, T-1$ **do**
    `// compute return-to-go`
    $G_t = \sum_{k=t+1}^{T} \gamma^{k-t-1} r_k$
    `// policy update`
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_t|s_t) G_t$
**until** convergence
**return** $\pi_{\boldsymbol{\theta}}$

### 4.2.4 Implementation Notes

- When computing return-to-go, we can iterate backward from the terminal time to start, since $G_t \leftarrow \gamma G_{t+1} + r_{t+1}$ where $t = 0, 1, 2, \ldots, T-1$.

### 4.2.5 Pros and Cons

Pros:

- No need for model;
- Can cover both discrete and continuous actions;

Cons:

- Large variance;
- May converge to suboptimal policy;

# Chapter 5

# Model-based Methods

# Chapter 6

# Comparisons of Algorithms

# Part II

# Theories

# Chapter 7

# Basic Concepts

## 7.1 Introduction

In this section, we discuss common concepts that are used frequently in reinforcement learning.

## 7.2 State, Action, Policy and Reward

### 7.2.1 State

The **state** is the status of the agent with respect to the environment. The set of all possible states, the **state space**, is denoted as $\mathcal{S}$.

### 7.2.2 Action

The **action** is what agent can do, when a state is given. By choosing the action and taking it, the agent moves to the next state, which is called **state transition**. The set of all possible actions in state $s$, **action space** for state $s$, is denoted as $\mathcal{A}(s)$.

#### 7.2.2.1 Transition Probability

Note that the transition can be stochastic. That is, the same choice of action in the same state may not lead to equivalent next state. We denote this probability with conditional porbability as following:

$$p(s'|s,a) \tag{7.1}$$

where $s', s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$.

### 7.2.3 Policy

The **policy** tells the agent which actions to take at every state. The policy can be deterministic or stochastic. We denote the probability of taking action $a$ under state $s$ following policy $\pi$ as $\pi(a|s)$.

### 7.2.4 Reward

**Reward** is one of the most unique concept of reinforcement learning. After the execution of action, the agent receives a reward, denoted as $r$. The agent's goal is to maximize the total reward that the agent receives. Due to this goal, the reward can encourage or discourage the agent's choice of selecting an action under certain state. Again, the reward can be stochastic, too.

One question that follows naturally is: Can't we just pick the actions that gives the most reward at every state? If this was correct, the reinforcement learning may not have existed. Since the reward is just an immediate information of the act, choosing only the action with highest reward is not the correct answer, most of the time. There can be a better choice in the long run.

## 7.3 Trajectories, Returns and Episodes

### 7.3.1 Trajectory

The **trajectory** is the footsteps of the agent. More formally, suppose that the agent is on the initial state $s_0$ and took action $a_0$. The agent receives reward $r_1$ from environment and moves to state $s_1$. Again, the agent take an action $a_1$, receive reward $r_1$ and move to state $s_2$. Repeating this, we get the trajectory of an agent:

$$s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_{T-1}, a_{T-1}, rT. \tag{7.2}$$

### 7.3.2 Returns

Given the trajectory $\tau = \{s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_{T-1}, a_{T-1}, rT\}$, we can calculate the **return** $G_t$, of the trajectory. The naive definition of the return, called **total rewards** is the sum

$$G_t = \sum_{k=t+1}^{T} r_k. \tag{7.3}$$

The point is that we do not know if the terminal time $T$ is finite or not. If $T$ is not finite, than the cummulative reward 7.3 diverges. For this, we introduce the **discounted return**, defined as

$$G_t = \sum_{k=t+1}^{\infty} \gamma^{k-t-1} r_k \tag{7.4}$$

where $\gamma \in [0, 1]$ is called **discount rate**. This works for for finite trajectory too, if we let $r_k = 0$ for $k > T$.

### 7.3.3 Episodes

The **episode** is the finite trajectory itself, like 7.2. The episode may stop at the states called **terminal states**. The tasks with episodes are called **episodic tasks**. Unlike episodic tasks, there are **continuing tasks**, which does not ends. These two can be expressed in unified mathematical manner. For this, in the episodic task, we must consider the agent's behavior after the terminal state: to stay in that state forever, or to take action like normal states. In this note, we follow the latter, like [Zha25].

## 7.4 Markov Decision Process

### 7.4.1 Definition

The **Markov decision process** (MDP) is the general framework that describes the stochastic dynamic systems. It is defined as following:

**Definition 7.4.1.1 (Markov Decision Process)**

The **Markov decision process** is the 5-tuple $(\mathcal{S}, \mathcal{A}, p, \mathcal{R}, \gamma)$, where each elements are defined as following:

- $\mathcal{S}$ is the state space, $\mathcal{A}(s)$ is the action space associated with state $s$, and $\mathcal{R}(s,a)$ is a set of rewards, associated with each state-action pair $(s,a)$.
- $p(s', r|s, a)$ is the dynamics of the model, which is as probability of moving to state $s'$ from $s$ by taking action $a$, and obtaining reward $r$. Note that the state transition probability and the reward probability can be decribed with dynamics. See 7.4.1.1
- $\pi(a|s)$ is the policy, a probability of agent choosing action $a$ in state $s$.
- $\gamma$ is the discount rate.

with the dynamics satisfying the *Markov property*:

$$p(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0, a_0) = p(s_{t+1}|s_t, a_t) \tag{7.5}$$

$$p(r_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0, a_0) = p(r_{t+1}|s_t, a_t) \tag{7.6}$$

where $t$ is the current time step.

**Remark 7.4.1.1 (Transition and reward probability as dynamics)**

The 4-tuple dynamics can be used to describe transition probability and reward probability as following:

$$p(s'|s,a) = \sum_{r \in \mathcal{R}(s,a)} p(s', r|s, a) \tag{7.7}$$

$$p(r|s,a) = \sum_{s' \in \mathcal{S}} p(s', r|s, a) \tag{7.8}$$

The dynamics can be either stationary or non-stationary. If the dynamics changes over time, it is non-stationary; else, it is called stationary.

# Chapter 8

# Bellman Equation

## 8.1 Introduction

TODO: Introduce the Bellman Operator and Bellman Optimality Operator. The **Bellman equation** is the heart of reinforcement learning. All reinforcement learning algorithms are designed to solve this equation.

## 8.2 Bellman Equation

### 8.2.1 State-value Function

Before diving into the Bellman equation, we introduce an important concept called the **value of a state**.

**Definition 8.2.1.1 (State Value)**

Let the agent follows the policy $\pi$. The **state value** $v_\pi(s)$ of state $s$ is defined as the expected return:

$$v_\pi(s) = \mathbb{E}[G_t | S_t = s] \tag{8.1}$$

where $G_t$ is the discounted return $G_t = \sum_{k=t+1}^{\infty} \gamma^{k-t-1} R_k$. The function $v_\pi$ itself is called **state-value function**.

State values are used to evaluate the policy $\pi$. Policies that generate greater state values are better. Then, how can we obtain te state values? The answer is to solve the Bellman equation.

### 8.2.2 Bellman Equation for State-Value Function

The Bellman equation is simply a set of linear equations that describe the relationships of each state values. Bellman equation can be obtained by using the recursive property of return:

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots && (8.2)\\ &= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4}) + \cdots && (8.3)\\ &= R_{t+1} + \gamma G_{t+1}. && (8.4) \end{aligned}$$

Substituting equation 8.1 with the recursive realationship, we obtain

$$v_\pi(s) = \mathbb{E}[G_t|S_t = s] \tag{8.5}$$
$$= \mathbb{E}[R_{t+1} + \gamma G_{t+1}|S_t = s] \tag{8.6}$$
$$= \mathbb{E}[R_{t+1}|S_t = s] + \gamma\mathbb{E}[G_{t+1}|S_t = s] \tag{8.7}$$

Using the law of total expectation 10.2, we get

$$\mathbb{E}[R_{t+1}|S_t = s] = \sum_{a\in\mathcal{A}(s)} \pi(a|s)\mathbb{E}[R_{t+1}|S_t = s, A_t = a] \tag{8.8}$$
$$= \sum_{a\in\mathcal{A}(s)} \pi(a|s) \sum_{r\in\mathcal{R}(s,a)} p(r|s,a)r \tag{8.9}$$

and

$$\mathbb{E}[G_{t+1}|S_t = s] = \sum_{s'\in\mathcal{S}} \mathbb{E}[G_{t+1}|S_{t+1} = s', S_t = s]p(s'|s) \tag{8.10}$$
$$= \sum_{s'\in\mathcal{S}} \mathbb{E}[G_{t+1}|S_{t+1} = s']p(s'|s) \quad \text{(Markov property)} \tag{8.11}$$
$$= \sum_{s'\in\mathcal{S}} v_\pi(s')p(s'|s) \tag{8.12}$$
$$= \sum_{s'\in\mathcal{S}} v_\pi(s') \sum_{a\in\mathcal{A}(s)} p(s'|s,a). \tag{8.13}$$

The final result is given in the following box:

**Theorem 8.2.2.1 (Bellman Equation for State-Value Function)**

Let $\pi$ be a policy, and let $v_\pi$ be a state-value function. Then, the following Bellman equation is satisfied:

$$v_\pi(s) = \sum_{a\in\mathcal{A}(s)} \pi(a|s) \sum_{s'\in\mathcal{S}} \sum_{r\in\mathcal{R}(s,a)} p(s', r|s, a)\big[r + \gamma v_\pi(s')\big] \tag{8.14}$$

### 8.2.3 Matrix-Vector form of Bellman Equation

We can reformulate Bellman equation in terms of matrix-vector form. For this, we rewrite the Bellman equation given in Theorem 8.2.2.1 as

$$v_\pi(s) = r_\pi(s) + \gamma \sum_{s'\in\mathcal{S}} p_\pi(s'|s)v_\pi(s'), \tag{8.15}$$

where

$$r_\pi(s) = \sum_{a\in\mathcal{A}(s)} \pi(a|s) \sum_{r\in\mathcal{R}(s,a)} p(r|s,a)r \tag{8.16}$$
$$p_\pi(s'|s) = \sum_{a\in\mathcal{A}(s)} \pi(a|s)p(s'|s,a). \tag{8.17}$$

The $r_\pi$ and $p_\pi$ denotes the mean of immediate rewards and probability of transitioning from $s$ to $s'$ under policy $\pi$, respectively. Suppose that we indexed all states as $s_i$ with $i = 1, \ldots, n$, where $n = |\mathcal{S}|$. Let

$$v_\pi = \begin{bmatrix} v_\pi(s_1) \\ \vdots \\ v_\pi(s_n) \end{bmatrix}, r_\pi = \begin{bmatrix} r_\pi(s_1) \\ \vdots \\ r_\pi(s_n) \end{bmatrix}, P_\pi = \begin{bmatrix} p_\pi(s_1|s_1) & p_\pi(s_2|s_1) & \ldots & p_\pi(s_n|s_1) \\ p_\pi(s_1|s_2) & p_\pi(s_2|s_2) & \ldots & p_\pi(s_n|s_2) \\ \vdots & \vdots & & \vdots \\ p_\pi(s_1|s_n) & p_\pi(s_2|s_n) & \ldots & p_\pi(s_n|s_n) \end{bmatrix}. \tag{8.18}$$

Then, Theorem 8.2.2.1 can be written in the following form:

**Theorem 8.2.3.1 (Matrix-Vector form of Bellman Equation)**

Let $v_\pi$, $r_\pi$ and $P_\pi$ be given as 8.18. Then, following is satisfied:

$$v_\pi = r_\pi + \gamma P_\pi v_\pi \tag{8.19}$$

**Remark 8.2.3.1 (Properties of Matrix $P_\pi$)**

Note that the matrix $P_\pi$ given in equation 8.18 satisfies following properties:

- All the elements are nonnegative;
- The sum of the values of every row equals 1. Matrix with this property is called a *stochastic matrix*.

## 8.3 Solving The Bellman Equation

There are two methods in solving the Bellman equation. The closed-form solution, and the iterative solution.

### 8.3.1 Closed-form Solution

Since we have represented the Bellman equation as the matrix form, solving it is very easy.

**Theorem 8.3.1.1 (Closed-form Solution of The Bellman Equation)**

Let $\pi$ be a given policy, and $v_\pi$, $r_\pi$, $P_\pi$ be given as 8.18. The solution to the equation $v_\pi = r_\pi + \gamma P_\pi v_\pi$ is given as

$$v_\pi = (I - \gamma P_\pi)^{-1} r_\pi \tag{8.20}$$

*Proof.* Manipulate the equation as following:

$$v_\pi = r_\pi + \gamma P_\pi v_\pi \tag{8.21}$$
$$v_\pi - \gamma P_\pi v_\pi = r_\pi \tag{8.22}$$
$$(I - \gamma P_\pi) v_\pi = r_\pi. \tag{8.23}$$

Now, what we have to show is that the matrix $I - \gamma P_\pi$ is invertible. We use the fact that $A$ is invertible $\Leftrightarrow$ eigenvalues of $A$ are nonzero. To show this, we use Geshgorin Circle Theorem 11.2.0.1. Note that the $i$-th Geshgorin circles has a center at $1 - \gamma p(s_i|s_i)$, with radius $\sum_{j \neq i} \gamma p_\pi(s_j|s_i)$. Since $P_\pi$ is stochastic matrix, we know that $\sum_j \gamma p(s_j|s_i) = \gamma < 1$. That is,

$\sum_{j \neq i} \gamma p_\pi(s_j|s_i) < 1 - \gamma p_\pi(s_i|s_i)$, which means that the $i$-th Geshgorin circle cannot contain the origin. According to the Geshgorin Circle Theorem, all the eigenvalues cannot be zero. □

TODO: add properties of $I - \gamma P_\pi$.

### 8.3.2 Iterative Solution

The direct method for solving the Bellman equation is not used very well in practice, since the inverse matrix computation is very costly. Instead of the close-form method, in practice, we use iterative method to solve the Bellman equation.

**Theorem 8.3.2.1 (Iterative Solution of The Bellman Equation)**

Let $\pi$ be a policy and $v_\pi$, $r_\pi$, and $P_\pi$ be that of 8.18. Then, the solution of 8.2.3.1 equals the limit of following sequence $\{v_k\}$:

$$v_{k+1} = r_\pi + \gamma P_\pi v_k \tag{8.24}$$

*Proof.* What we have to show is that $v_k \to v_\pi$ as $k \to \infty$. For this, we let $\delta_k = v_k - v_\pi$ and show that $\delta_k \to 0$. Substituting $v_{k+1} = \delta_{k+1} + v_\pi$ and $v_k = \delta_k + v_\pi$ into equation 8.24, we have

$$\delta_{k+1} = \gamma P_\pi \delta_k. \tag{8.25}$$

Since $P_\pi$ is a stochastic matrix and $0 < \gamma < 1$, we know that $\delta_k \to 0$ as $k \to \infty$. □

## 8.4 Bellman Equation for Action-Value Function

Now we introduce the **action value**, which denotes the value of an action at a state.

### 8.4.1 Action Value

The action value is defined as following:

**Definition 8.4.1.1 (Action Value)**

Let $\pi$ be a given policy. Then, the **action value** of a state-action pair is defined as

$$q_\pi(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]. \tag{8.26}$$

**Remark 8.4.1.1 (Properties of Action Value)**

Action value $q_\pi(s, a)$ has following properties.

- $v_\pi = \sum_{a \in \mathcal{A}(s)} \pi(a|s) q_\pi(s, a)$
- $q_\pi(s, a) = \sum_{r \in \mathcal{R}(s,a)} p(r|s, a) r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) v_\pi(s')$

*Proof.*
- Use the law of total expectation 10.2.

- Use the result above and 8.2.2.1.

□

### 8.4.2 Bellman Equation for Action-Value Function

The Bellman equation for action-value function is already in our hand, since actually the Remark 8.4.1.1 is what all we need.

**Theorem 8.4.2.1 (Bellman Equation for Action-value Function)**

Let $\pi$ be a given policy, and let $q_\pi(s,a)$ be an action value. Then, the following Bellman equation is satisfied:

$$q_\pi(s,a) = \sum_{r\in\mathcal{R}(s,a)} p(r|s,a)r + \gamma \sum_{s'\in\mathcal{S}} p(s'|s,a) \sum_{a'\in\mathcal{A}(s')} \pi(a'|s')q_\pi(s',a') \tag{8.27}$$

$$= \sum_{r\in\mathcal{R}(s,a)} p(r|s,a)r + \gamma \sum_{s'\in\mathcal{S}} p(s'|s,a)v_\pi(s') \tag{8.28}$$

Moreover, the matrix-vector form is given as

$$q_\pi = \tilde{r} + \gamma \tilde{P}\Pi q_\pi \tag{8.29}$$

where $[q_\pi]_{(s,a)} = q_\pi(s,a)$, $\tilde{r}_{(s,a)} = \sum_{r\in\mathcal{R}(s,a)} p(r|s,a)r$, $[\tilde{P}]_{(s,a),s'} = p(s'|s,a)$ and $\Pi$ is a block diagonal matrix such that $\Pi_{s',(s',a')} = \pi(s'|a')$ for $a' \in \mathcal{A}(s')$ and other entries are all zero.

*Proof.* Use the Remark 8.4.1.1. For the matrix-vector form, try to derive it, as we did in Theorem 8.2.3.1. □

# Chapter 9

# Bellman Optimality Equation

## 9.1 Introduction

This chapter introduces the Bellman optimal equation (BOE), and proves serveral properties of it. This section mainly follows the book [Zha25].

## 9.2 Optimal Policy

The reinforcement learning algorithms are all designed to find an optimal policy for given enviroment. Then, what is an 'optimal' policy? The definition is given below.

**Definition 9.2.0.1 (Optimal Policy and Optimal State Value)**

A policy $\pi^*$ is optimal if and only if $v_\pi^*(s) \geq v_\pi(s)$ for all $s \in \mathcal{S}$ and for any other policy $\pi$. The state values of $\pi^*$ are the optimal state values.

**Definition 9.2.0.2 (Better Policy)**

A policy $\pi_1$ is **better** than a policy $\pi_2$ if and only if $v_{\pi_1}(s) \geq v_{\pi_2}(s)$ for all $s \in \mathcal{S}$.

The questions that naturally arise are:

- Existence: Does the optimal policy exists?
- Uniqueness: Is the optimal policy unique?
- Stochasticity: Is the optimal policy stochastic or deterministic?
- Algorithm: How do we obtain the optimal policy and the optimal state values?

## 9.3 Bellman Optimality Equation

The optimal policy and optimal state values can be analysed via the **Bellman optimality equation** (BOE). The optimal state values satisfies following Bellman optimality equation. This claim in fact has two logically independent statements:

1. The BOE, viewed purely as a fixed-point equation, has a unique solution (Section 9.4);
2. The unique solution of BOE concides with the optimal stat-value function $v^*$, and the derived greedy policy is optimal (Section 9.5)

Note that the Section 9.4 does not assume that the state-value function is optimal.

**Definition 9.3.0.1 (Bellman Optimality Equation)**

Let $\Pi$ be a set of possible policies and let $\gamma \in (0,1)$ be a discount rate. Then, the Bellman Optimality Equation is defined as

$$v(s) = \max_{\pi \in \Pi} \sum_{a \in \mathcal{A}(s)} \pi(a|s) \left( \sum_{r \in \mathcal{R}} p(r|s,a) r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s,a) v(s') \right) \tag{9.1}$$

$$= \max_{\pi \in \Pi} \sum_{a \in \mathcal{A}(s)} \pi(a|s) q(s,a) \tag{9.2}$$

where $v(s), v(s')$ are unkown variables to be solved. Additionally, the matrix-vector form of Bellman optimality equation is given as

$$v = \max_{\pi \in \Pi} (r_\pi + \gamma P_\pi v) \tag{9.3}$$

where $r_\pi$ and $P_\pi$ are given as 8.18.

Using the matrix-vector form of BOE, we can express BOE very simply as following:

$$v = f(v) \tag{9.4}$$

where $f(v) = \max_{\pi \in \Pi}(r_\pi + \gamma P_\pi v)$ .

## 9.4 Solving The Bellman Optimality Equation

### 9.4.1 Contraction Mapping Theorem

We propose a **contraction mapping theorem**, the main tool for solving the BOE. The contraction mapping theorem is a powerful tool for analyzing general nonlinear equations.

**Definition 9.4.1.1 (Contraction Mapping)**

Let $f : \mathbb{R}^d \to \mathbb{R}^d$. Then, the function $f$ is a **contraction mapping** if there exists $\gamma \in (0,1)$ such that

$$\|f(x_1) - f(x_2)\| \leq \gamma \|x_1 - x_2\| \tag{9.5}$$

for all $x_1, x_2 \in \mathbb{R}^d$.

Now we present the contraction mapping theorem.

**Theorem 9.4.1.1 (Contraction Mapping Theorem)**

Let $f : \mathbb{R}^n \to \mathbb{R}^n$ be a contraction mapping. Then, the following properties hold for equation $x = f(x)$:

- Existence: There exists a fixed point $x^*$ satisfying $f(x^*) = x^*$;
- Uniqueness: The fixed point $x^*$ is unique;
- Algorithm: The following sequence $\{x_k\}$ converges to the fixed point $x^*$ exponentially, for any initial $x_0$.

$$x_{k+1} = f(x_k) \tag{9.6}$$

*Proof. Part 1.* (Convergence) The sequence $\{x_k\}$ defined as $x_{k+1} = f(x_k)$ converges.

We show that the sequence $\{x_k\}$ is a Cauchy sequence; i.e., there always exists an $N \in \mathcal{N}$ such that $\|x_m - x_n\| < \epsilon$ for any $\epsilon > 0$. The Cauchy sequence has a nice property; the Cauchy sequence always converges. This can be proved by using the properties of $\mathcal{R}$, that the $\mathcal{R}$ is a complete set. We won't show this here. The following holds:

$$\begin{aligned}
\|x_{k+1} - x_k\| =& \|f(x_k) - f(x_{k-1})\| && (9.7)\\
\leq& \gamma\|x_k - x_{k-1}\| && (9.8)\\
\leq& \gamma^2\|x_{k-1} - x_{k-2}\| && (9.9)\\
\leq& \gamma^3\|x_{k-2} - x_{k-3}\| && (9.10)\\
&\vdots && (9.11)\\
\leq& \gamma^k\|x_1 - x_0\| && (9.12)
\end{aligned}$$

Using this, we can show that

$$\begin{aligned}
\|x_m - x_n\| &\leq \|x_m - x_{m-1}\| + \|x_{m-1} - x_{m-2}\| + \cdots + \|x_{n+1} - x_n\| && (9.13)\\
&\leq \gamma^{m-1}\|x_1 - x_0\| + \gamma^{m-2}\|x_1 - x_0\| + \cdots + \gamma^n\|x_1 - x_0\| && (9.14)\\
&= \|x_1 - x_0\|(\gamma^{m-1} + \gamma^{m-2} + \cdots + \gamma^n) && (9.15)\\
&= \|x_1 - x_0\|\frac{\gamma^n(1 - \gamma^{m-n-2})}{1 - \gamma} && (9.16)\\
&\leq \|x_1 - x_0\|\frac{\gamma^n}{1 - \gamma} && (9.17)
\end{aligned}$$

where $m \geq n$, without loss of generality. Since $\gamma \in (0, 1)$, the last line converges to 0. That is, the sequence $\{x_k\}$ is a Cauchy sequence.

*Part 2.* (Existence) $\lim_{k\to\infty} x_k$ is a solution of fixed-point equation $x = f(x)$.

Let $x^* = \lim_{k\to\infty} x_k$. Then, following holds.

$$\begin{aligned}
\|x^* - f(x^*)\| &\leq \|x^* - x_k\| + \|f(x_k) - f(x^*)\| + \|x_k - f(x_k)\| && (9.18)\\
&\leq \|x^* - x_k\| + \gamma\|x_k - x^*\| + \|x_k - f(x_k)\| && (9.19)
\end{aligned}$$

Using the results of *Part 1* and squeeze theorem, we can show that $x^* = f(x^*)$.

*Part 3.* (Uniqueness) The fixed-point $x^*$ is unique.

Suppose there is another fixed point $x'$. Then, $\|x^* - x'\| = \|f(x^*) - f(x')\| \leq \gamma\|x^* - x'\|$. Since $\gamma \in (0, 1)$, the only possibility is $x^* = x'$.

□

### 9.4.2 Contraction Property of Bellman Optimality Equation

Now, to use the contraction mapping theorem, we prove that the right-hand side of BOE 9.3 is a contraction mapping.

**Theorem 9.4.2.1 (Contraction Property of Bellman Optimality Equation)**

Let the function $f$ be that of equation 9.3. Then, the following contraction mapping property holds for any $v_1, v_2 \in \mathbb{R}^{|\mathcal{S}|}$:

$$\|f(v_1) - f(v_2)\|_\infty \leq \gamma\|v_1 - v_2\|_\infty \tag{9.20}$$

where $\gamma \in (0, 1)$ is the discount rate, and $\|\|_\infty$ is the maximum norm, which is the maximum absolute value of the elements in a vector.

*Proof.* Let $v_1, v_2 \in \mathbb{R}^{\mathcal{S}}$, and let $\pi_1^* = \arg\max_\pi (r_\pi + \gamma P_\pi v_1)$, $\pi_2^* = \arg\max_\pi (r_\pi + \gamma P_\pi v_2)$. Then,

$$f(v_1) = \max_\pi (r_\pi + \gamma P_\pi v_1) = r_{\pi_1^*} + \gamma P_{\pi_1^*} v_1 \geq r_{\pi_2^*} + \gamma P_{\pi_2^*} v_1 \tag{9.21}$$

$$f(v_2) = \max_\pi (r_\pi + \gamma P_\pi v_2) = r_{\pi_2^*} + \gamma P_{\pi_2^*} v_2 \geq r_{\pi_1^*} + \gamma P_{\pi_1^*} v_2 \tag{9.22}$$

where $\geq$ is applied elementwise. This result leads to

$$f(v_1) - f(v_2) \leq \gamma P_{\pi_1^*}(v_1 - v_2) \tag{9.23}$$

$$f(v_2) - f(v_1) \leq \gamma P_{\pi_2^*}(v_2 - v_1) \tag{9.24}$$

Let $z = \max\{|\gamma P_{\pi_1^*}(v_1 - v_2)|, |\gamma P_{\pi_2^*}(v_2 - v_1)|\}$, where $\max(\cdot)$ and $|\cdot|$ are all applied elementwise. Then,

$$\|f(v_1) - f(v_2)\|_\infty \leq \|z\|_\infty. \tag{9.25}$$

Now, let $z_i$ be the $i$-th entry of $z$, and let $p_i^\top$ and $q_i^\top$ be the $i$-th row of $P_{\pi_1^*}$ and $P_{\pi_2^*}$, respectively. Then, $z_i = \max\{\gamma|p_i^\top(v_1 - v_2)|, \gamma|q_i^\top(v_1 - v_2)|\}$. Since all the elements of vector $p_i$ is nonnegative and thier sum equals 1,

$$|p_i^\top(v_1 - v_2)| \leq p_i^\top|v_1 - v_2| \leq \|v_1 - v_2\|_\infty. \tag{9.26}$$

where $|\cdot|$ is applied elementwise. Similarly, $|q_i^\top(v_1 - v_2)| \leq \|v_1 - v_2\|_\infty$. Therefore $\|z\|_\infty \leq \gamma\|v_1 - v_2\|_\infty$. Substituting to 9.25,

$$\|f(v_1) - f(v_2)\|_\infty \leq \gamma\|v_1 - v_2\|_\infty. \tag{9.27}$$

□

## 9.5 Optimal Policy, Optimal State Value and Bellman Optimality Equation

We have a following Theorem of BOE:

**Theorem 9.5.0.1 (Solution of the Bellman Optimality Equation)**

Let $f$ be as the equation 9.3. Then, the BOE $v = f(v)$ has following properties:

1. There exists a unique solution;
2. The solution can be obtained iteratively by $v_{k+1} = f(v_k)$.

*Proof.* All the previous Theorems (9.4.1.1, 9.4.2.1) proves this trivially. □

Now, we should prove that the solution of the Bellman optimality equation is really an optimal state-value function, and prove that the greedy policy derived from it is the optimal policy.

**Theorem 9.5.0.2 (Optimality of $v^*$ and $\pi^*$)**

Let $v^*$ be the solution of BOE $v = f(v)$, and let $\pi^* = \arg\max_{\pi\in\Pi}(r_\pi + \gamma P_\pi v^*)$. The, the $v^*$ is an optimal state vaule function, and the $\pi^*$ is an optimal policy. That is, for any policy $\pi$, following holds;

$$v^* = v_{\pi^*} \geq v_\pi \tag{9.28}$$

where $v_\pi$ is the state value function of $\pi$, and $\geq$ is applied elementwise.

*Proof.* For any policy $\pi$, $v_\pi = r_\pi + \gamma P_\pi v_\pi$ holds (Bellman equation). Then,

$$\begin{aligned} v^* &= \max_{\tau\in\Pi}(r_\tau + \gamma P_\tau v^*) && (9.29)\\ &= r_{\pi^*} + \gamma P_{\pi^*} v^* && (9.30)\\ &\geq r_\pi + \gamma P_\pi v^* && (9.31) \end{aligned}$$

and we have

$$v^* - v_\pi \geq \gamma P_\pi (v^* - v_\pi). \tag{9.32}$$

Using this inequality repeatedly, we have $v^* - v_\pi \geq \gamma^n P_\pi^n (v^* - v_\pi)$ for any $n \in \mathbb{N}$. Since $\gamma \in (0, 1)$ and $P_\pi$ is a nonnegative matrix with all its elements less than or equal to 1, $\lim_{n\to\infty} \gamma^n P_\pi^n (v^* - v_\pi) = 0$. That is, $v^* - v_\pi \geq \lim_{n\to\infty} \gamma^n P_\pi^n (v^* - v_\pi) = 0$. □

The following theorem examines the optimal policy more precisely.

**Theorem 9.5.0.3 (Greedy Optimal Policy)**

For any $s \in \mathcal{S}$, the following deterministic policy is an optimal policy for solving the BOE.

$$\pi^*(a|s) = \begin{cases} 1, & a = \arg\max_a q^*(a,s) \\ 0, & a \neq \arg\max_a q^*(a,s) \end{cases} \tag{9.33}$$

where $q^*(s,a) = \sum_{r\in\mathcal{R}} p(r|s,a)r + \gamma \sum_{s'\in\mathcal{S}} p(s'|s,a)v^*(s')$.

*Proof.* From Theorem 9.5.0.1 and 9.3.0.1, we see that

$$v^*(s) = \max_{\pi\in\Pi} \sum_{a\in\mathcal{A}} \pi(a|s) q^*(s,a) \tag{9.34}$$

$$\leq \left( \max_{\pi\in\Pi} \sum_{a\in\mathcal{A}} \pi(a|s) \right) \max_{a'} q^*(s,a') \tag{9.35}$$

$$= \max_{a'} q^*(s,a') \tag{9.36}$$

and the given deterministic policy $\pi^*$ satisfies the equality of inequality 9.35. □

The important truth is that the optimal state value is unique but the optimal policy is not. The Theorem 9.5.0.3 only shows that there always exists at least one deterministic optimal policy. There still can be other optimal policies, too, stochastic or whatever.

## 9.6 Factors that Influence Optimal Policies

Now, we analyze the factors of BOE that influences optimal policies. The factors are: reward and discount rate.

### 9.6.1 Reward

### 9.6.2 Discount Rate

# Part III

# Appendices

# Chapter 10

# Probability Theory

## 10.1 Introduction

A short, useful theorems of probability theory.

## 10.2 Law of Total Expectation

The followinga are the law of total expectations.

$$\mathbb{E}[X] = \sum_{a} \mathbb{E}[X|A = a]p(a) \tag{10.1}$$

$$\mathbb{E}[X|A = a] = \sum_{b} \mathbb{E}[X|A = a, B = b]p(b|a) \tag{10.2}$$

# Chapter 11

# Linear Algebra

## 11.1 Introduction

A short useful theorems of linear algebra.

## 11.2 Geshgorin Circle Theorem

**Theorem 11.2.0.1 (Geshgorin Circle Theorem)**

# Bibliography

[Gra20] Laura Graesser. *Foundations of deep reinforcement learning. Theory and practice in Python*. Ed. by Wah Loon Keng. Addison Wesley data & analytics series. Includes bibliographical references and index. Boston: Addison-Wesley, 2020. 379 pp. ISBN: 0135172381.

[Sut20] Richard S. Sutton. *Reinforcement learning. An introduction*. Ed. by Andrew Barto. Second edition. Adaptive computation and machine learning. Hier auch später erschienene, unveränderte Nachdrucke. Cambridge, Massachusetts: The MIT Press, 2020. 526 pp. ISBN: 9780262039246.

[Zha25] Shiyu Zhao. *Mathematical foundations of reinforcement learning*. [Tsinghua]: Tsinghua University Press, 2025. 1275 pp. ISBN: 9789819739448.